Object-Oriented Programming in Java

Java Generics

Exercises

8 hands-on coding exercises on the Generics slide deck (J05), numbered as Part 6: generic classes and methods, bounds, wildcards, erasure, generic functional interfaces and comparators. You build your own generic list and sorted table, then use them in a complete mock exam. Difficulty rises from ★ to ★★★.

Contents

How to use this sheet **2**

Part 6 – Generics (J05) **2**
6.1 ★ Generic pairs 2
6.2 ★★ Generic methods on arrays 3
6.3 ★★ Your own generic list 4
6.4 ★★ Your own sorted registry 5
6.5 ★★★ Generic functional helpers 6
6.6 ★★★ Comparator factories, by hand and from the library 7
6.7 ★★★ Wildcards and erasure 8
6.8 ★★★ Mock exam: a library 10

How to use this sheet

8 coding exercises on the Generics slide deck (J05), numbered as Part 6 after the 28 exercises on J01–J04, and getting harder: ★ warm-up, ★★ core, ★★★ challenge. Solutions are in the companion PDF *Java Generics – Solutions*, and the theory each exercise needs is in *Java Generics – Theory notes*.

The exercises follow the exam style of Parts 2–5: one package per exercise (`ex6_1`, ...), exact formats and messages, checked exceptions, and a `Main` whose body is the code before each sample run. Since the collections are not in the slides yet, 6.3 and 6.4 have you build your own generic list (`SimpleList`) and sorted table (`Registry`), with the method names of the library's `List` and, mostly, `Map` (`keys()` plays the role of `keySet()`); the later exercises reuse them, so import them from their packages. 6.8 is a complete mock exam, about two hours, with exam-style tests in the solutions.

Use Java 25. When the demo calls methods that throw a checked exception without catching it, the text says *main declared with: throws* ....

Exercise	Topics
6.1	generic classes, several type parameters, the diamond, generic methods, static factories, `equals` in a generic class
6.2	generic methods on arrays, bounded types, `Comparable<? super T>`, `Comparator<? super T>`, `Predicate`, no `new T[]`, array covariance
6.3	a generic container on `Object[]`, unchecked casts, `@SafeVarargs`, generic `Iterable`/`Iterator`, an API like `List`
6.4	bounded type parameters on a class, static nested generic classes, binary search, an API like `Map`, non-generic exceptions
6.5	`Function`, `UnaryOperator`, `BinaryOperator`, `Predicate`, `Consumer`, `BiConsumer`, `Supplier`, `ToDoubleFunction`, `and`/`or`/`negate`/`andThen`/`compose`, an anonymous class with state, grouping
6.6	`Comparator.comparing`, `comparingInt`, `reversed`, `thenComparing`, writing them by hand, overflow, top-n
6.7	invariance, `? extends`, `? super`, `?`, PECS, several bounds (`&`), type erasure, array generators
6.8	a mock exam: everything above, plus an enum and exam-style tests

Part 6 – Generics (J05)

6.1 ★ Generic pairs

Package `ex6_1`.

1. The slides' `Pair<T>`, with `first()`, `second()`, `set1st(T x)` and `set2nd(T x)`. Add:
   - `static <T> Pair<T> of(T a, T b)`: a generic factory method;
   - `boolean contains(T x)`: true if one of the two elements equals `x`; elements may be `null` (`Objects.equals`);
   - `Pair<T> swapped()`: a new pair with the elements exchanged;
   - `toString()` → `(One, Two)`.
2. `Couple<A, B>`, with two type parameters:
   - `first()` and `second()`;
   - `Couple<B, A> swap()`;
   - `<C> Couple<A, C> withSecond(C value)`: a generic method of a generic class;

- `toString()` → `<Rossi, 42>`;
- `equals` and `hashCode` on both elements.

3. `Scores`, a utility class whose methods return their results as pairs:
   - `static Couple<String, Integer> best(String[] names, int[] scores)`: the highest score, the first one on a tie; `IllegalArgumentException` if the arrays are empty or have different lengths;
   - `static Pair<Integer> minMax(int... values)`: the smallest and the largest value; `IllegalArgumentException` with no values.

```
Pair<String> sp = Pair.of("One", "Two");
Pair<Integer> ip = new Pair<>(4, 2);
String s = sp.second();            // no cast
int i = ip.first();                // auto-unboxing
IO.println(sp + " " + s + " " + i + " " + ip.swapped() + " " + sp.contains("One") + " "
        + ip.contains(3));
ip.set1st(10);
IO.println(ip + " " + (ip.first() + ip.second()));
Couple<String, Integer> c = new Couple<>("Rossi", 42);
Couple<Integer, String> r = c.swap();
Couple<String, Double> d = c.withSecond(27.5);
IO.println(c + " " + r + " " + d + " " + c.equals(new Couple<>("Rossi", 42)));
IO.println(Scores.best(new String[]{"Ann", "Bob", "Zoe"}, new int[]{7, 9, 9}) + " "
        + Scores.minMax(5, -2, 8, 3));
```

Sample run:

```
(One, Two) Two 4 (2, 4) true false
(10, 2) 12
<Rossi, 42> <42, Rossi> <Rossi, 27.5> true
<Bob, 9> (-2, 8)
```

6.2 ★★ Generic methods on arrays

Package `ex6_2`. Write a utility class `ArrayUtils` with these static generic methods:

- `<T> int indexOf(T[] a, T x)`: the first position equal to `x` (`null` allowed), or −1;
- `<T> T remove(T[] a, T el)`: the slides' method. It sets the first element equal to `el` to `null` and returns it, or returns `null`;
- `<T> void swap(T[] a, int i, int j)`;
- `<T extends Comparable<? super T>> T max(T[] a)`: the largest element, skipping `null`s; `null` for an empty array;
- `<T> T max(T[] a, Comparator<? super T> order)`: thanks to `? super T`, the maximum of an `Integer[]` found with a `Comparator<Object>` is still an `Integer`;
- `<T> T[] append(T[] a, T x)`: a new array, one element longer. `new T[...]` does not compile; use `Arrays.copyOf`;
- `<T> int count(T[] a, Predicate<? super T> cond)`;
- `<T> T[] filter(T[] a, Predicate<? super T> cond)`: a new array of the same type with the elements that satisfy `cond`.

Test them with `record Product(String name, double price) implements Comparable<Product>`, whose natural order is by price.

```
String[] words = {"fox", "dog", "cat", "dog"};
Integer[] nums = {13, 8, 21, 5};
IO.println(ArrayUtils.indexOf(words, "dog") + " " + ArrayUtils.indexOf(nums, 99) + " "
        + ArrayUtils.max(words) + " " + ArrayUtils.max(nums));
String removed = ArrayUtils.remove(words, "dog");     // no cast
IO.println(removed + " " + Arrays.toString(words));
ArrayUtils.swap(nums, 0, 3);
IO.println(Arrays.toString(ArrayUtils.append(nums, 34)) + " "
        + ArrayUtils.count(nums, n -> n % 2 == 1));
Product[] ps = {
    new Product("pen", 1.5), new Product("bag", 25.0), new Product("ink", 4.0)
};
IO.println(ArrayUtils.max(ps) + " "
        + ArrayUtils.max(ps, Comparator.comparing(Product::name)));
Product[] cheap = ArrayUtils.filter(ps, p -> p.price() < 5);
IO.println(Arrays.toString(cheap) + " " + cheap.getClass().getSimpleName());
Comparator<Object> byText = Comparator.comparing(Object::toString);
Integer m = ArrayUtils.max(nums, byText);             // T = Integer: see the notes
IO.println(m);
Object[] objects = words;                             // arrays are covariant...
try {
    objects[0] = 42;                                  // ...so this compiles
} catch (ArrayStoreException e) {
    IO.println("ArrayStoreException: " + e.getMessage());
}
```

Sample run:

```
1 -1 fox 21
dog [fox, null, cat, dog]
[5, 8, 21, 13, 34] 3
Product[name=bag, price=25.0] Product[name=pen, price=1.5]
[Product[name=pen, price=1.5], Product[name=ink, price=4.0]] Product[]
8
ArrayStoreException: java.lang.Integer
```

6.3 ★★ Your own generic list

Package `ex6_3`. `SimpleList<E> implements Iterable<E>` is a growable list. Its methods have the same names and meaning as those of `java.util.List`, so that switching to the library's `ArrayList`, when the collections arrive, costs nothing. (Only `equals` and `hashCode` stay those of `Object`.)

- The elements live in an `Object[]`, because you cannot create an `E[]`. The initial capacity is 4, and the array doubles when it is full (`Arrays.copyOf`). Add a package-private `int capacity()` for the demo.
- `boolean add(E e)` (always `true`), `void add(int index, E e)`, `E get(int index)`, `E set(int index, E e)` (returns the old element), `E remove(int index)`, `boolean remove(Object o)`, `int size()`, `boolean isEmpty()`, `boolean contains(Object o)`, `int indexOf(Object o)` (`null` allowed) and `void clear()`.
- A bad index throws `IndexOutOfBoundsException` with the message `index 5, size 3`.
- `void sort(Comparator<? super E> order)`: sort the occupied part of the array with `Arrays.sort(…, 0, size, order)`, as `ArrayList` does, so the sort is stable; `null` means natural order. The call needs a cast: see the theory notes.
- `iterator()`: a private inner class, whose `next()` throws `NoSuchElementException` at the end.
- `toString()` → `[a, b, c]`.
- `@SafeVarargs static <E> SimpleList<E> of(E... items)`.

```
SimpleList<String> names = new SimpleList<>();
names.add("Zoe");
names.add("Ann");
names.add("Bob");
names.add(1, "Eva");
IO.println(names + " size=" + names.size() + " " + names.get(1) + " "
        + names.indexOf("Bob") + " " + names.contains("Max"));
for (int i = 0; i < 3; i++) {
    names.add("X" + i);
}
IO.println(names.size() + " " + names.capacity());
IO.println(names.remove(0) + " " + names.remove("X1") + " " + names.remove("nobody")
        + " " + names);
names.sort(Comparator.naturalOrder());
IO.println(names);
String initials = "";
for (String n : names) {
    initials += n.charAt(0);
}
IO.println(initials);
SimpleList<Integer> nums = SimpleList.of(5, 3, 8);
nums.set(0, nums.get(0) * 10);
int total = 0;
for (int n : nums) {
    total += n;
}
IO.println(nums + " total=" + total);
try {
    nums.get(3);
} catch (IndexOutOfBoundsException e) {
    IO.println("error: " + e.getMessage());
}
```

Sample run:

```
[Zoe, Eva, Ann, Bob] size=4 Eva 3 false
7 8
Zoe true false [Eva, Ann, Bob, X0, X2]
[Ann, Bob, Eva, X0, X2]
ABEXX
[50, 3, 8] total=61
error: index 3, size 3
```

6.4 ★★ Your own sorted registry

Package `ex6_4` (it uses `ex6_3.SimpleList`). `Registry<K extends Comparable<? super K>, V>` is a table from keys to values, kept sorted by key; it plays the role of the exam's `Map`s.

- The entries are objects of a private static nested generic class `Entry<K, V>`, kept in an array sorted by key. Find a key with binary search, and insert by shifting the following entries.
- `V put(K key, V value)`: adds or replaces, and returns the old value or `null`, as `Map.put` does.
- `void add(K key, V value) throws DuplicateKeyException`: only for new keys (message `duplicate key <key>`).
- `V get(K key)` (`null` if absent), `V getOrDefault(K key, V otherwise)`, and `V require(K key) throws MissingKeyException` (message `missing key <key>`).
- `boolean containsKey(K key)`, `V remove(K key)` (returns the removed value or `null`), `int size()`.
- `SimpleList<K> keys()` and `SimpleList<V> values()`, in key order.

- `toString()` → `{bag=3, cap=1}`.
- The two exceptions are normal checked exceptions: an exception class cannot be generic.

```
// main declared with: throws DuplicateKeyException, MissingKeyException
Registry<String, Integer> stock = new Registry<>();
IO.println(stock.put("pen", 10) + " " + stock.put("ink", 4) + " " + stock.put("pen", 12));
for (String w : "bag pen bag cap bag".split(" ")) {
    stock.put(w, stock.getOrDefault(w, 0) + 1);
}
IO.println(stock + " size=" + stock.size());
IO.println(stock.keys() + " " + stock.values() + " " + stock.get("zip") + " "
        + stock.containsKey("cap"));
try {
    stock.add("ink", 1);
} catch (DuplicateKeyException e) {
    IO.println("error: " + e.getMessage());
}
try {
    stock.require("zip");
} catch (MissingKeyException e) {
    IO.println("error: " + e.getMessage());
}
Registry<Integer, String> byYear = new Registry<>();
byYear.put(2017, "Delivery");
byYear.put(2014, "Trail");
byYear.put(2016, "Restaurants");
IO.println(byYear + " " + byYear.remove(2014) + " " + byYear.keys());
```

Sample run:

```
null null 10
{bag=3, cap=1, ink=4, pen=13} size=4
[bag, cap, ink, pen] [3, 1, 4, 13] null true
error: duplicate key ink
error: missing key zip
{2014=Trail, 2016=Restaurants, 2017=Delivery} Trail [2016, 2017]
```

6.5 ★★★ Generic functional helpers

Package `ex6_5`. A utility class `Fun` works on `SimpleList` (6.3) and `Registry` (6.4), the way you will soon work with the library's streams. The exam's reports, such as "for each category, the list of…", are built exactly like `groupBy`.

- `<T, R> SimpleList<R> map(SimpleList<T> list, Function<? super T, ? extends R> f);`
- `<T> SimpleList<T> filter(SimpleList<T> list, Predicate<? super T> cond);`
- `<T> T reduce(SimpleList<T> list, T identity, BinaryOperator<T> op);`
- `<T> void forEach(SimpleList<T> list, Consumer<? super T> action);`
- `<T> double sum(SimpleList<T> list, ToDoubleFunction<? super T> value);`
- `<T, K extends Comparable<? super K>> Registry<K, SimpleList<T>> groupBy(SimpleList<T> list, Function<? super T, ? extends K> key);`
- `<T, K extends Comparable<? super K>> Registry<K, Integer> countBy(SimpleList<T> list, Function<? super T, ? extends K> key);`
- `<T> SimpleList<T> generate(int n, Supplier<? extends T> next)`: the results of `n` calls of `next.get()`.

Test them with `record Sale(String product, String category, int qty, double price)`. The

demo also uses the default methods of the library's functional interfaces (`and`, `or`, `negate`, `andThen`, `compose`), `UnaryOperator`, `BiConsumer`, and a `Supplier` written as an anonymous class, which keeps a counter like the code generators of the exams.

```
SimpleList<Sale> sales = SimpleList.of(
        new Sale("pen", "office", 10, 1.5),
        new Sale("ink", "office", 2, 4.0),
        new Sale("apple", "food", 6, 0.5),
        new Sale("bread", "food", 1, 2.25),
        new Sale("bag", "travel", 1, 25.0));
Predicate<Sale> office = x -> x.category().equals("office");
Predicate<Sale> big = x -> x.qty() >= 5;
IO.println(Fun.map(Fun.filter(sales, office.and(big)), Sale::product) + " "
        + Fun.map(Fun.filter(sales, office.negate().or(big)), Sale::product));
Function<Sale, Double> amount = x -> x.qty() * x.price();
Function<Sale, String> label = amount.andThen(a -> String.format(Locale.US, "%.2f", a));
IO.println(Fun.map(sales, label));
Function<String, Integer> length = String::length;
IO.println(Fun.map(sales, length.compose(Sale::product)));
IO.println(Fun.reduce(Fun.map(sales, Sale::qty), 0, Integer::sum) + " "
        + Fun.sum(sales, x -> x.qty() * x.price()));
Registry<String, SimpleList<Sale>> byCategory = Fun.groupBy(sales, Sale::category);
BiConsumer<String, Integer> show = (k, n) -> IO.print(k + ":" + n + " ");
for (String k : byCategory.keys()) {
    show.accept(k, byCategory.get(k).size());
}
IO.println();
IO.println(Fun.countBy(sales, x -> x.price() < 2 ? "cheap" : "normal"));
UnaryOperator<String> shout = w -> w.toUpperCase() + "!";
Fun.forEach(Fun.map(Fun.filter(sales, big), Sale::product),
        p -> IO.print(shout.apply(p) + " "));
IO.println();
Supplier<String> codes = new Supplier<>() {     // an object with its own state
    private int last = 0;

    @Override
    public String get() {
        return "S" + (++last);
    }
};
IO.println(Fun.generate(3, codes) + " " + codes.get());
```

Sample run:

```
[pen] [pen, apple, bread, bag]
[15.00, 8.00, 3.00, 2.25, 25.00]
[3, 3, 5, 5, 3]
20 53.25
food:2 office:2 travel:1
{cheap=2, normal=3}
PEN! APPLE!
[S1, S2, S3] S4
```

6.6 ★★★ Comparator factories, by hand and from the library

Package `ex6_6`. The slides show how `Comparator.comparing` and its friends are written. Write them yourself in a class `Comparators`, with the right wildcards:

- `static <T, U extends Comparable<? super U>> Comparator<T> comparing(Function<? super T, ? extends U> key);`

- `static <T> Comparator<T> comparingInt(ToIntFunction<? super T> key)`, with `Integer.compare`. The slides subtract, which overflows; see the demo;
- `static <T> Comparator<T> reverse(Comparator<T> order)`;
- `static <T> Comparator<T> then(Comparator<T> first, Comparator<? super T> second)`;
- `static <T> SimpleList<T> top(SimpleList<T> list, int n, Comparator<? super T> order)`: the first `n` elements in that order, as a new list (the exam's "best customers"), or all of them if there are fewer; `IllegalArgumentException` if `n` is negative. The original list must not change.

Test them with `record Student(int id, String last, String first, double average)`. In the demo class, write two helpers:

- `static <T> SimpleList<T> sorted(SimpleList<T> list, Comparator<? super T> order)`: a sorted copy;
- `static String names(SimpleList<Student> list)`: the first names, separated by spaces.

```
SimpleList<Student> st = SimpleList.of(
        new Student(11, "Rossi", "Carla", 27.5),
        new Student(7, "Rossi", "Alice", 27.5),
        new Student(3, "Bianchi", "Bruno", 29.0),
        new Student(-2_000_000_000, "Neri", "Ugo", 18.0),
        new Student(2_000_000_000, "Verdi", "Eva", 30.0));
Comparator<Student> mine = Comparators.then(Comparators.comparing(Student::last),
        Comparators.comparing(Student::first));
Comparator<Student> lib = Comparator.comparing(Student::last)
        .thenComparing(Student::first);
IO.println(names(sorted(st, mine)) + " | " + names(sorted(st, lib)));
IO.println(names(sorted(st, Comparators.comparingInt(Student::id))) + " | "
        + names(sorted(st, (a, b) -> a.id() - b.id())));
IO.println(names(Comparators.top(st, 2,
        Comparators.reverse(Comparator.comparingDouble(Student::average)))));
IO.println(names(Comparators.top(st, 3, Comparator.comparingDouble(Student::average)
        .reversed().thenComparing(Student::last).thenComparing(Student::first))));
Comparator<Object> byText = Comparator.comparing(Object::toString);
// a Comparator<Object> is a Comparator<? super Integer>
IO.println(Comparators.top(SimpleList.of(10, 9, 100), 3, byText));
IO.println(names(st));
```

Sample run:

```
Bruno Ugo Alice Carla Eva | Bruno Ugo Alice Carla Eva
Ugo Bruno Alice Carla Eva | Eva Ugo Bruno Alice Carla
Eva Bruno
Eva Bruno Alice
[10, 100, 9]
Carla Alice Bruno Ugo Eva
```

6.7 ★★★ Wildcards and erasure

Package `ex6_7` (it uses `SimpleList` of 6.3 and `Pair` of 6.1). Write a utility class `Lists`:

- `static double sum(SimpleList<? extends Number> list)`;
- `static <T> void copy(SimpleList<? super T> dst, SimpleList<? extends T> src)`: appends the elements of `src` to `dst`; `copy(a, a)` doubles `a`;
- `@SafeVarargs static <T> void addAll(SimpleList<? super T> dst, T... items)`;
- `static String describe(SimpleList<?> list)` → `3 elements: [a, b, c]`;

- `static <T extends Comparable<? super T>> boolean isSorted(SimpleList<T> list)`;
- `static <T extends Comparable<? super T>> T max(SimpleList<? extends T> list)`: `NoSuchElementException` for an empty list, as `Collections.max`;
- `static <T extends Number & Comparable<? super T>> double range(SimpleList<T> list)`: the largest element minus the smallest, both found with `compareTo`, as a `double` (`doubleValue`). A type parameter with two bounds; `NoSuchElementException` for an empty list;
- `static <T> void sortPair(Pair<T> p, Comparator<? super T> order)`: the slides' pair sorting;
- `static <T> T[] toArray(SimpleList<T> list, IntFunction<T[]> maker)`. Since `new T[n]` does not compile, the caller passes a way to create the array, as in `toArray(list, String[]::new)`.

```
SimpleList<Integer> ints = SimpleList.of(3, 1, 2);
SimpleList<Double> doubles = SimpleList.of(0.5, 0.25);
IO.println(Lists.sum(ints) + " " + Lists.sum(doubles));
IO.println(Lists.range(ints) + " " + Lists.range(doubles));
SimpleList<Number> numbers = new SimpleList<>();
Lists.copy(numbers, ints);                    // Integer -> Number
Lists.copy(numbers, doubles);                 // Double  -> Number
Lists.addAll(numbers, 7, 8.5);
IO.println(Lists.describe(numbers) + " sum=" + Lists.sum(numbers));
SimpleList<Object> objects = new SimpleList<>();
Lists.addAll(objects, "x", 1, 'c');
IO.println(Lists.describe(objects));
IO.println(Lists.isSorted(ints) + " " + Lists.isSorted(SimpleList.of("a", "b", "b")) + " "
        + Lists.max(ints));
Pair<Integer> p = new Pair<>(4, 2);
Comparator<Number> byValue = (a, b) -> Double.compare(a.doubleValue(), b.doubleValue());
Lists.sortPair(p, byValue);   // a Comparator<Number> sorts a Pair<Integer>
IO.println(p);
String[] arr = Lists.toArray(SimpleList.of("b", "a"), String[]::new);
IO.println(arr.length + " " + arr[0] + " " + arr.getClass().getSimpleName());
SimpleList<String> strings = new SimpleList<>();
IO.println((strings instanceof SimpleList<?>) + " "
        + (strings.getClass() == ints.getClass()));
```

Sample run:

```
6.0 0.75
2.0 0.25
7 elements: [3, 1, 2, 0.5, 0.25, 7, 8.5] sum=22.25
3 elements: [x, 1, c]
false true 3
(2, 4)
2 b String[]
true true
```

6.8 ★★★ Mock exam: a library

Package `ex6_8`. This is a complete exam in the style of the past ones, to do in about two hours. The facade is `Library`; all errors are `LibraryException`, and only the methods written with `throws LibraryException` declare it. Use `SimpleList` (6.3) and `Registry` (6.4) wherever the exam solutions use `List` and `Map`. The solutions also contain the tests `TestR1_Books` ... `TestR4_Reports`, written like the exam's: as at the exam, run them after each requirement.

R1 – Books.

- `String addBook(String title, String author, int year, int copies) throws LibraryException` returns the codes `B1`, `B2`, ... Error if a book with the same title and author exists (compared exactly), or if `copies` < 1.
- `String getBook(String code) throws LibraryException` → `B1 Dune - Herbert (1965)`; error for an unknown code.
- `SimpleList<String> findByAuthor(String author)`: the codes of that author's books (author ignoring case), by year and then by title.

R2 – Members.

- `int addMember(String name)` → 1, 2, ... Two members may have the same name.
- `String getMember(int id) throws LibraryException` → `1 Mario Rossi`; error for an unknown id.

R3 – Loans.

- `int lend(String code, int memberId) throws LibraryException` returns the loan ids 1, 2, ... Error if the book or the member does not exist, no copy of the book is available, or the member already has 3 active loans.
- `void giveBack(int loanId) throws LibraryException`: error if the loan does not exist or is already returned.
- `int available(String code) throws LibraryException`: the copies not on loan; error for an unknown code.
- `LoanStatus status(int loanId) throws LibraryException`, with `enum LoanStatus { ACTIVE, RETURNED }`; error for an unknown loan.

R4 – Reports.

- `SimpleList<String> mostLent(int n)`: the `n` books lent most often (returned loans included), as `B2:3`, by decreasing count and then by code number (`B2` before `B10`). Books never lent are left out.
- `Registry<String, SimpleList<String>> activeLoans()`: for each name of a member with active loans, the titles they have now, sorted; members with the same name share one entry.
- `Registry<Integer, Integer> titlesPerDecade()`: for each decade (1960, 1970, ...), how many books were published in it.

```
// main declared with: throws LibraryException
Library lib = new Library();
String dune = lib.addBook("Dune", "Herbert", 1965, 2);
String found = lib.addBook("Foundation", "Asimov", 1951, 1);
String robots = lib.addBook("I, Robot", "Asimov", 1950, 1);
String neu = lib.addBook("Neuromancer", "Gibson", 1984, 1);
int mario = lib.addMember("Mario Rossi");
int anna = lib.addMember("Anna Bianchi");
IO.println(lib.getBook(dune) + " | " + lib.findByAuthor("asimov") + " | "
        + lib.getMember(anna));
int l1 = lib.lend(dune, mario);
int l2 = lib.lend(dune, anna);
int l3 = lib.lend(found, anna);
try {
    lib.lend(dune, mario);
} catch (LibraryException e) {
    IO.println("error: " + e.getMessage());
}
lib.giveBack(l1);
int l4 = lib.lend(dune, mario);
lib.lend(neu, mario);
IO.println(l1 + " " + l2 + " " + l3 + " " + l4 + " " + lib.status(l1) + " "
        + lib.status(l4) + " " + lib.available(dune));
IO.println(lib.mostLent(3));
IO.println(lib.activeLoans());
IO.println(lib.titlesPerDecade());
try {
    lib.giveBack(l1);
} catch (LibraryException e) {
    IO.println("error: " + e.getMessage());
}
```

Sample run (the messages are free; these are the ones in the solution):

```
B1 Dune - Herbert (1965) | [B3, B2] | 2 Anna Bianchi
error: no copy of B1 available
1 2 3 4 RETURNED ACTIVE 0
[B1:3, B2:1, B4:1]
{Anna Bianchi=[Dune, Foundation], Mario Rossi=[Dune, Neuromancer]}
{1950=2, 1960=1, 1980=1}
error: loan 1 already returned
```